CSci 130 – Web Programming
Wordle Web Application Project Specification

Undergraduate CSci 130 Web Programming Project

Students will design and implement a complete multi-page Wordle website. The game logic must run in JavaScript in the browser, while the server supplies the five-letter word dataset and securely stores accounts, game results, cumulative statistics, and leaderboard data.

Project element	Required implementation
Client	HTML, CSS, and JavaScript Wordle interface and game logic
Server	Word dataset API, authentication, statistics, and leaderboard endpoints
Database	Users, game sessions, and persistent player statistics
Pages	Home, rules, creator, game, login or registration, and leaderboard
Core limit	Six guesses for each five-letter target word

Submission deadline: See Deadline on Canvas (Monday of the last week of instruction)

Team size: 2

1 Project Purpose

The purpose of this project is to integrate client-side programming, asynchronous communication, server-side development, database design, authentication (using the code provided in class), validation, and usable interface design in one application. The finished website must support complete gameplay for guests and persistent statistics for authenticated players.

2 Learning Objectives

- Construct a responsive multi-page website using semantic HTML and maintainable CSS.
- Implement a non-trivial interactive application with JavaScript and DOM events.
- Use fetch and JSON to exchange data between the browser and server to update the leaderboard and authenticate users
- Design server routes and a relational database schema for accounts and game records.
- Apply password hashing, sessions or tokens, authorization, and server-side validation.
- Compute aggregate statistics and sortable leaderboard results from persistent data.
- Test normal behavior, boundary cases, invalid input, and common security failures.

3 Required Technology and Architecture

You will use PHP on the server side. Web frameworks are not allowed. You will use Apache and the DBMS that was presented in class. The following separation of responsibility is required.

Layer	Responsibilities	Restrictions
Browser client	Render pages, manage the current board, accept keyboard and mouse input, evaluate letters, display feedback, track elapsed time, and call server APIs.	The Wordle game must be implemented in client-side JavaScript. A third-party Wordle package may **not** replace the game logic.
Web server	Serve static assets and the word dataset; register and authenticate users; validate completed games; persist results; return player statistics and leaderboard data.	Passwords and database credentials must never be sent to or stored in client-side code!!
Database	Store users, game sessions or results, and the fields needed to calculate cumulative statistics.	Do not store plaintext passwords. Use constraints and parameterized queries or an ORM.

4 Required Pages and Navigation

Every page must use a consistent header, navigation menu, visual identity, and footer. The current page should be visually identifiable. The layout must work on common desktop and mobile widths.

Page	Required content and behavior
Home	Project title, short introduction, clear link or button to play, and navigation to all public pages.
Rules	Objective, six-guess limit, valid-guess requirement, color meanings, repeated-letter behavior, scoring table, and end-of-game conditions.

Page	Required content and behavior
Creator	Creator name (group members), brief biography or project statement, role, and contributions, and optional professional or course-related links.
Game	Six-row board, five columns per row, physical and on-screen keyboard support, status messages, timer, current score opportunity, new-game control, and end-of-game summary.
Login and registration	Account creation, login, logout, validation feedback, and clear authenticated state. These may be separate pages or one combined page.
Leaderboard	Sortable player table with the required cumulative measures. Sorting must be available through accessible controls or clickable column headers.

5 Word Dataset

- The server must expose a dataset containing five-letter words through an HTTP endpoint. The browser must load the dataset asynchronously; it may not be embedded directly in the JavaScript source.
- Normalize entries to one case, remove duplicates, and reject entries that are not exactly five alphabetic letters. The application must handle an unavailable or malformed dataset with a useful error message.
- The dataset must support both target-word selection and guess validation. If the instructor provides separate answer and allowed-guess lists, the implementation must preserve that distinction.
- A new target must be selected without bias. Repetition across games is allowed unless the instructor requires a no-repeat policy.

6 Game Rules and Client-Side Behavior

6.1 Game sequence

1. Load and validate the five-letter word dataset from the server.
2. Select one target word and initialize a board with six rows of five cells.
3. Accept letters, Backspace, and Enter from the physical keyboard and an on-screen keyboard.
4. Permit submission only when the active row has five letters and the guess exists in the allowed dataset.
5. If the submission does not belong to the list of words, you can decrease the score by 32.
6. Evaluate the guess, display feedback, update the keyboard, and move to the next row when needed.
7. End the game immediately after a correct guess or after the sixth incorrect guess.
8. Show the target word, result, points earned, elapsed time, and a control to begin a new game.

6.2 Letter evaluation

Each submitted letter must receive one of three states: correct letter and position, present elsewhere in the word, or absent. Repeated letters must be evaluated using letter counts: first mark exact matches, then mark misplaced matches only while unused occurrences remain in the target. A letter may not be credited more times than it occurs in the target word.

6.3 Scoring

A successful game begins with a possible score of 1024 points. The possible score is divided by two for every additional trial. A game that is not solved after six trials earns 0 points.

Trial on which word is solved	Points earned
1	1024
2	512
3	256
4	128
5	64
6	32
Not solved	0

Required formula for a win on trial n: score = 1024 / 2^(n - 1), where 1 <= n <= 6. Cumulative score is the sum of points earned in all completed games.

6.4 Timing

Timing begins when a new target and playable board are ready. Timing ends when the game is won or the sixth guess is completed. The game duration must be submitted to the server in seconds. The server must validate that the value is numeric, nonnegative, and within an instructor-approved maximum. Total time played is the sum of durations for completed games.

7 Accounts and Authentication

- Registration must require a unique username and a password. An email address may be included if required by the instructor.
- Passwords must be hashed with a modern password-hashing algorithm such as Argon2, bcrypt, or scrypt. Plaintext passwords must never be logged or stored. See PHP example provided on Canvas.
- Authentication must use a server-managed session with a secure cookie or a correctly implemented token mechanism. Protected endpoints must reject unauthenticated requests.
- Registration, login, and logout must provide clear success and error feedback without revealing whether sensitive account information exists beyond what is necessary.
- All input must be validated on the server even if the browser also performs validation. Database access must use parameterized.

8 Statistics and Persistence

Authenticated players must retain their results across browser sessions and devices. Guest users may play, but their results do not need to be added to the persistent leaderboard. The following definitions must be used consistently in the user interface, API, and database.

Statistic	Definition
Cumulative score	Sum of points earned from all completed games; unsuccessful games add zero.

Statistic	Definition
Games played	Number of games started by the player, including a current unfinished game if the design persists starts.
Completed games	Number of games that reached a terminal state: a win or the sixth unsuccessful guess.
Games won	Number of completed games in which the target word was found.
Total time played	Sum of elapsed seconds for completed games only.

At minimum, the authenticated game page must display cumulative score, games played, games won, and total time played. Completed games must also be stored because it is a required leaderboard field. Abandoned games must not be counted as completed and must not add score or completed-game time.

9 Leaderboard

The leaderboard must display one row per player and support sorting by each required measure. The active sort column and direction must be visible. Sorting may occur on the client for a fully loaded dataset or on the server through query parameters.

Column	Default display	Required sorting
Player	Username	Alphabetical, optional
Score	Integer points	Ascending and descending
Games played	Integer	Ascending and descending
Total time	Readable duration such as 01:24:35	Ascending and descending by numeric seconds
Completed games	Integer	Ascending and descending

The default ordering should be score descending. Recommended tie-breakers are completed games descending, total time ascending, then username ascending. Pagination is recommended for large datasets but is not required unless specified by the instructor.

10 Minimum Data Model

Students may extend the schema, but the database must preserve sufficient information to audit statistics and prevent a player from being credited more than once for the same game.

Entity	Minimum fields
User	user_id primary key; username unique; password_hash; created_at
Game	game_id primary key; user_id foreign key; started_at; completed_at nullable; status; attempts_used nullable; score default 0; duration_seconds nullable

Recommended status values are in_progress, won, lost, and abandoned. Aggregate statistics may be calculated from game rows or cached in a user-statistics table. If cached values are used, updates must be atomic so game history and totals cannot disagree.

11 Minimum API Contract

Route names may differ, but equivalent functionality is required. All request and response bodies should use JSON except for static files. Use appropriate HTTP status codes.

Method and route	Purpose	Access
GET /api/words	Return the validated five-letter word dataset or the instructor-approved word-list representation.	Public
POST /api/auth/register	Create an account after validating username and password.	Public
POST /api/auth/login	Authenticate and establish a session or return a token.	Public
POST /api/auth/logout	End the authenticated session.	Authenticated
GET /api/me	Return the current user and cumulative statistics.	Authenticated
POST /api/games/start	Create a game record and return its identifier.	Authenticated
POST /api/games/:id/complete	Record terminal result, attempts, score, and duration exactly once.	Authenticated owner
GET /api/leaderboard	Return player rows using validated sort and direction parameters.	Public

Because target selection and evaluation occur in client-side JavaScript, a determined user can inspect or alter client data. For this CSci 130 course project, the server must still recompute the score from attempts, validate ownership and state transitions, reject duplicate completion, and never trust a client-supplied cumulative total. A production competitive game would require stronger server authority over target selection and guesses.

12 Validation and Error Handling

- Reject guesses that are not exactly five alphabetic letters or are absent from the allowed list.
- Prevent input while animations or asynchronous completion requests are pending.
- Prevent guesses after a terminal state and prevent more than six submitted guesses.
- Handle word-list, authentication, statistics, and leaderboard failures without crashing or leaving the page in a misleading state.
- Reject invalid leaderboard sort fields and directions rather than interpolating them into a query.
- Prevent duplicate game completion and modifications to another user's game.
- Preserve correct repeated-letter feedback for targets and guesses with duplicate letters.

13 Accessibility and Interface Quality

- Use semantic landmarks, descriptive page titles, visible keyboard focus, and logically ordered headings.
- Allow gameplay and leaderboard sorting without a mouse.

- Do not communicate letter state through color alone; include a shape, label, pattern, or accessible text announcement.
- Provide sufficient color contrast and readable text at mobile and desktop sizes.
- Use an ARIA live region or equivalent mechanism for invalid guesses, results, and status changes when appropriate.
- Do not display the target word before the game ends or include it in ordinary visible page content.

14 Testing Requirements

Submit a concise test plan and evidence that the implementation was tested. You can provide screenshots of the working application.

Area	Minimum cases
Scoring	Win on trials 1 through 6; loss after trial 6; cumulative total.
Letter evaluation	All correct; all absent; misplaced letters; repeated letters in guess; repeated letters in target.
Input	Too short; too long; nonletters; word absent from list; keyboard and on-screen keyboard.
Authentication	Valid registration and login; duplicate username; incorrect password; logout; protected route without login.
Persistence	Statistics remain after refresh and a new login; completion cannot be recorded twice.
Leaderboard	Every required sort field in both directions; numeric time sorting; deterministic tie behavior.
Resilience	Server unavailable; word dataset unavailable; malformed response; database error handled safely.

15 Required Deliverables

- Complete source code for the client, server, database schema or migrations, and configuration examples.
- README with prerequisites, installation, database setup, environment variables, run commands, and test instructions.
- A seed or import procedure for the five-letter dataset.
- Database design documentation, including an entity-relationship diagram or equivalent schema description. Provide a script to create automatically the tables.
- Test plan and test results covering the minimum cases in this specification.
- A short demonstration or presentation showing registration, gameplay, score calculation, persistence, and leaderboard sorting (to happen during the lab sessions of the last week).

16 Suggested Development Milestones

Milestone	Expected result
1 Interface foundation	Shared navigation and responsive Home, Rules, Creator, Game, Login, and Leaderboard pages.

Milestone	Expected result
2 Client game	Dataset fetch, six-row board, input, validation, repeated-letter evaluation, timer, and scoring.
3 Accounts and database	Schema, registration, hashed passwords, login, logout, and authenticated state.
4 Persistence and leaderboard	Game start and completion records, statistics, leaderboard query, and sortable table.
5 Quality assurance	Security review, accessibility improvements, automated or documented tests, and final documentation.

17 Grading Rubric

Category	Points	Evidence
Client-side Wordle implementation	25	Correct six-trial gameplay, word validation, duplicate-letter logic, keyboard interaction, score, timer, and end states.
Server and word dataset	10	Reliable server delivery of validated five-letter words and appropriate error handling.
Authentication and security	15	Registration, login, logout, password hashing, protected routes, validation, and safe database access.
Database and statistics	15	Sound schema; correct persistence of games, wins, completed games, score, and total time; duplicate protection.
Leaderboard	10	Required columns, correct aggregates, sorting in both directions, and understandable time display.
Pages, design, and accessibility	10	All required pages, consistent navigation, responsive layout, keyboard access, contrast, and non-color status cues.
Code quality	5	Clear organization, meaningful names, modular functions, minimal duplication, and useful comments.
Testing and error handling	5	Evidence for required cases and graceful handling of client, server, and data failures.
Documentation and demonstration	5	Complete setup instructions, schema and API documentation, dataset citation, and functional demonstration.

Total: 100 points. Instructors (lab and class) may apply additional penalties for an application that cannot be installed, exposes plaintext passwords, contains copied game logic without attribution, or fails to include required source files.

18 Acceptance Checklist

- ☐ All required pages are present and linked through consistent navigation.
- ☐ The browser fetches a server-hosted five-letter word dataset.
- ☐ The JavaScript game enforces five letters and no more than six guesses.
- ☐ Repeated letters are evaluated correctly.
- ☐ The scoring sequence is 1024, 512, 256, 128, 64, and 32; a loss earns 0.
- ☐ Accounts persist in a DBMS and passwords are securely hashed.
- ☐ Authenticated results update score, played, completed, won, and total time correctly.
- ☐ The leaderboard sorts by score, games played, total time, and completed games.
- ☐ Server validation prevents duplicate completion and unauthorized game updates.
- ☐ The site is responsive, keyboard usable, and understandable without relying only on color.
- ☐ README, schema or migrations, API documentation, dataset citation, and tests are included.